

M04 - Python Foundations

16-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Write, debug and explain foundational Python programs independently.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M00 setup discipline. SQL knowledge helps later, but this module starts Python syntax from first principles.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE04-01 - Python setup and first program	5
DE04-02 - Variables, data types and operators	7
DE04-03 - Strings and formatting	9
DE04-04 - Conditions and boolean logic	11
DE04-05 - Lists, tuples, sets and dictionaries	13
DE04-06 - Loops and iteration	16
DE04-07 - Functions and scope	18
DE04-08 - Modules, imports and project structure	20
DE04-09 - Exceptions and debugging basics	22
DE04-10 - Files, paths, CSV and JSON I/O	24
DE04-11 - pandas foundations: load, inspect, filter and aggregate	27
DE04-12 - Practical Big-O intuition	30
DE04-13 - Coding drills: lists and strings	32
DE04-14 - Coding drills: dictionaries and sets	34
DE04-15 - Stack, queue, search and sort basics	36
DE04-16 - Integrated Python checkpoint and reproducible script habits	39

Prerequisite, tool and outcome map

ASSUMED

Complete M00 setup discipline. SQL knowledge helps later, but this module starts Python syntax from first principles.

MODULE PURPOSE

Write, debug and explain foundational Python programs independently.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE04-01 - Python setup and first program

1. Why it matters

Run a Python program from a clean start, distinguish code from output, and use an error message as evidence instead of guessing. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Run a Python program from a clean start, distinguish code from output, and use an error message as evidence instead of guessing.

3. Prerequisite / what you already know

Complete M00 setup discipline. SQL knowledge helps later, but this module starts Python syntax from first principles.

4. Picture it in your head

Python is an interpreter executing a script from top to bottom. Picture a tiny machine: source file in, statements executed, output/error out.

5. New words before we use them

Python is an interpreter: it reads valid Python instructions and evaluates them according to the language rules. A program can be one line or many files. For a beginner, the safest loop is tiny change -> run -> inspect output/error -> explain what happened.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:08:42–0:18:52

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=wUSDVGivd-8))

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
print("
Hello, data engineering"
) print(2 + 3) # Change one value, run again, and compare the output. Run both lines. Change the message. Create one
simple syntax error on purpose, read the exception/line indicator, repair only that defect, then run again from a clean start.
```

8. Expected result / what you should see

Expected output appears; the error is preserved in your notes; the repaired file runs after closing and reopening the interpreter/editor.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Python setup and first program: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

If nothing runs, first prove which Python executable you are using (``py --version`` or ``python --version``). If syntax fails, read the last error line and inspect quotes, parentheses and indentation near the reported location.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Python is an interpreter executing a script from top to bottom. 3. Run the earliest independent control: For Python setup and first program: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

- With the video closed, create ``p01_first_program.py`` that prints your name, ``Standalone DE 2026``, and the result of `7 *`
- Create and repair one syntax error, preserving the first failed output.

13. Validate your result

For Python setup and first program: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Python setup and first program in your own words. Start with the purpose: Run a Python program from a clean start, distinguish code from output, and use an error message as evidence instead of guessing. Then name one realistic failure mode and explain how you would prove the result using this check: For Python setup and first program: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Python setup and first program with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-02.

DE04-02 - Variables, data types and operators

1. Why it matters

Represent identifiers, counts, money-like values, status text, booleans and missing values with intentional Python types and valid operators. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Represent identifiers, counts, money-like values, status text, booleans and missing values with intentional Python types and valid operators.

3. Prerequisite / what you already know

DE04-01 mastered or explicitly remediated.

4. Picture it in your head

A variable is a labeled box pointing to a value; the data type describes what operations make sense on that value. Operators combine values according to those type rules.

5. New words before we use them

A variable is a name bound to a value. The value has a type, and the type affects which operations are meaningful. ``4`` is an integer; ``850.0`` is a floating-point number; ``"O2001"`` is text even though it contains digits; ``True`/`False`` are booleans; and ``None`` represents the absence of a Python value.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:18:52–0:48:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
order_id = "O2001"
```

```
qty = 4 unit_price = 850.0 gross = qty * unit_price is_large = gross > 3000
```

print(order_id, gross, is_large, type(gross)) Create examples of int, float, str, bool and None. Print each value and its type. Then calculate `qty * unit_price` and one boolean comparison.

8. Expected result / what you should see

Types match business meaning; arithmetic is applied only to measures; comparison output is boolean; you can explain why an identifier remains text.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Variables, data types and operators: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Print the value and ``type(value)`` before changing code. If concatenation/arithmetic fails, inspect whether text needs parsing or whether the field should remain text.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A variable is a labeled box pointing to a value; the data type describes what operations make sense on that value. 3. Run the earliest independent control: For Variables, data types and operators: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create `order_id="O3107"`, `qty=4`, `unit_price=850.0`, `status="Completed"`. Calculate gross sales and a boolean for `gross >= 3000`. Explain the type of every variable.

13. Validate your result

For Variables, data types and operators: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Types match business meaning; arithmetic is applied only to measures; comparison output is boolean; you can explain why an identifier remains text.

14. Teach it back in your own words

Without notes, explain Variables, data types and operators in your own words. Start with the purpose: Represent identifiers, counts, money-like values, status text, booleans and missing values with intentional Python types and valid operators. Then name one realistic failure mode and explain how you would prove the result using this check: For Variables, data types and operators: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Variables, data types and operators with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-03.

DE04-03 - Strings and formatting

1. Why it matters

Clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning.

3. Prerequisite / what you already know

DE04-02.

4. Picture it in your head

A string is an ordered sequence of characters. Formatting is building a final message by inserting values into a controlled template rather than manually concatenating fragile fragments.

5. New words before we use them

Strings are sequences of characters. Common cleaning methods such as `.strip()`, `.lower()`, `.upper()` and `.title()` return new strings; they do not magically define the correct business value. Cleaning should therefore be deliberate and auditable.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:48:41–1:05:06

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`raw = " NORTH zone " clean = raw.strip().title() print(repr(raw)) print(f"Region: {clean}")` Apply strip/lower/upper/title to sample labels and compare ``repr`` before/after. Build two f-string labels.

8. Expected result / what you should see

Whitespace changes are visible; casing is intentional; original can still be compared; f-string output matches the required label exactly.

9. Why it worked

This works because clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning is made explicit before the result is accepted. Your independent check is: For Strings and formatting: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

If output looks identical, print ``repr(value)``. If title-case changes a name incorrectly, stop using it as a blanket business rule and define an explicit mapping instead.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A string is an ordered sequence of characters. 3. Run the earliest independent control: For Strings and formatting: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Normalize " EAST hub " to "East Hub" and create ``Hub: East Hub | Status: Ready``. Preserve and print the raw value too.

13. Validate your result

For Strings and formatting: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Whitespace changes are visible; casing is intentional; original can still be compared; f-string output matches the required label exactly.

14. Teach it back in your own words

Without notes, explain Strings and formatting in your own words. Start with the purpose: Clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning. Then name one realistic failure mode and explain how you would prove the result using this check: For Strings and formatting: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Strings and formatting with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-04.

DE04-04 - Conditions and boolean logic

1. Why it matters

Translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests.

3. Prerequisite / what you already know

DE04-03.

4. Picture it in your head

An if/elif/else chain is a decision tree: each condition is a branch, and only the first matching branch should control the intended action.

5. New words before we use them

A condition is an expression whose result is truthy/falsey; explicit comparisons usually make business rules easier to audit. `and` requires both sides to be true, while `or` requires at least one. Parentheses are useful when a rule combines several conditions.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 1:26:20–1:34:24

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
status = "Completed" amount = 2500 if status == "Completed" and amount >= 2500: label = "priority" else: label = "standard" print(label)
```

Test Completed/2800, Completed/2499, Completed/2500 and Pending/3000. State the expected label before running.

8. Expected result / what you should see

Boundary cases behave exactly as the written rule; a high Pending amount does not become priority.

9. Why it worked

This works because translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests is made explicit before the result is accepted. Your independent check is: For Conditions and boolean logic: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Write the English rule in one sentence, circle the words AND/OR, then translate one clause at a time. Print intermediate booleans if needed.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An if/elif/else chain is a decision tree: each condition is a branch, and only the first matching branch should control the intended action. 3. Run the earliest independent control: For Conditions and boolean logic: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify an order as `priority` only if status is Completed and amount ≥ 2500 ; otherwise `standard`. Use at least four boundary cases.

13. Validate your result

For Conditions and boolean logic: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Boundary cases behave exactly as the written rule; a high Pending amount does not become priority.

14. Teach it back in your own words

Without notes, explain Conditions and boolean logic in your own words. Start with the purpose: Translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests. Then name one realistic failure mode and explain how you would prove the result using this check: For Conditions and boolean logic: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Conditions and boolean logic with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-05.

DE04-05 - Lists, tuples, sets and dictionaries

1. Why it matters

Choose list, tuple, set or dict based on order, mutability, uniqueness/membership and key-value lookup needs. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Choose list, tuple, set or dict based on order, mutability, uniqueness/membership and key-value lookup needs.

3. Prerequisite / what you already know

DE04-04.

4. Picture it in your head

Picture Python collections as different containers: list = ordered shelf, tuple = fixed record, set = unique-members box, dictionary = key-to-value lookup cabinet.

5. New words before we use them

A list is an ordered mutable sequence. A tuple is an ordered sequence commonly used for a fixed-size record or values that should not be casually changed. A set stores unique hashable values and is excellent for membership/duplicate detection.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 1:34:24–2:12:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
pipeline = ["
extract"
, "
transform"
, "
load"
] coords = (23.3, 85.3) ids = {"
O1"
, "
O2"
, "
O1"
} manager = {"
East"
: "
Asha"
, "
West"
: "
Rohit"
} print(pipeline, coords, ids, manager["
East"
```

)] Build one list, tuple, set and dict from a small operations example. For each, say what property made it appropriate.

8. Expected result / what you should see

Choices match the requirement; duplicates in the set collapse intentionally; dictionary keys are meaningful/unique.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Lists, tuples, sets and dictionaries: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Ask four questions: do I need order? mutation? unique membership? key-to-value lookup? Choose the simplest structure that answers the requirement.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture Python collections as different containers: list = ordered shelf, tuple = fixed record, set = unique-members box, dictionary = key-to-value lookup cabinet. 3. Run the earliest independent control: For Lists, tuples, sets and dictionaries: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Map three regions to managers with a dict; deduplicate order IDs with a set; keep a three-step pipeline in a list; store one fixed coordinate pair in a tuple.

13. Validate your result

For Lists, tuples, sets and dictionaries: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Choices match the requirement; duplicates in the set collapse intentionally; dictionary keys are meaningful/unique.

14. Teach it back in your own words

Without notes, explain Lists, tuples, sets and dictionaries in your own words. Start with the purpose: Choose list, tuple, set or dict based on order, mutability, uniqueness/membership and key-value lookup needs. Then name one realistic failure mode and explain how you would prove the result using this check: For Lists, tuples, sets and dictionaries: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lists, tuples, sets and dictionaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-06.

DE04-06 - Loops and iteration

1. Why it matters

Iterate through collections, maintain explicit accumulators/counters and validate a loop with an independent control. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Iterate through collections, maintain explicit accumulators/counters and validate a loop with an independent control.

3. Prerequisite / what you already know

DE04-05.

4. Picture it in your head

A loop is a conveyor belt applying the same controlled step to one item at a time. The loop body, stopping condition, and accumulated state must be visible.

5. New words before we use them

A `for` loop visits items from an iterable one at a time. State that changes across iterations - such as a running total, count or output list - should be initialized before the loop and updated at the correct indentation level. While learning, a clear loop is often better than compressing everything into one line.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 2:21:06–2:36:31

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

values = [1700, 1350, 1100] total = 0 for value in values: total += value print(total) Total a list manually, count values >= 1500 and collect values below 1000. Only after the loop, compare the total with `sum()`.

8. Expected result / what you should see

Each input is processed once; accumulator starts at the correct value; control total matches; output list contains only eligible values.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Loops and iteration: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Print the accumulator inside the loop to see each step. Check indentation and initialization before changing the algorithm.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A loop is a conveyor belt applying the same controlled step to one item at a time. 3. Run the earliest independent control: For Loops and iteration: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From [1700,1350,1100,850,1800], compute total, count ≥ 1500 , and collect values below 1000 using a loop.

13. Validate your result

For Loops and iteration: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Each input is processed once; accumulator starts at the correct value; control total matches; output list contains only eligible values.

14. Teach it back in your own words

Without notes, explain Loops and iteration in your own words. Start with the purpose: Iterate through collections, maintain explicit accumulators/counters and validate a loop with an independent control. Then name one realistic failure mode and explain how you would prove the result using this check: For Loops and iteration: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Loops and iteration with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-07.

DE04-07 - Functions and scope

1. Why it matters

Write small reusable functions with explicit parameters, returned values, predictable behavior and no accidental global dependencies. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Write small reusable functions with explicit parameters, returned values, predictable behavior and no accidental global dependencies.

3. Prerequisite / what you already know

DE04-06.

4. Picture it in your head

A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change.

5. New words before we use them

A function gives a name to reusable logic. Parameters are inputs supplied by the caller; local variables exist inside the function; `return` sends a value back to the caller. `print` only displays something and is not a substitute for returning data needed by later code.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 2:52:12–3:02:03

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=wUSDVGivd-8))

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`def gross_sales(qty, unit_price): return qty * unit_price`
`print(gross_sales(2, 850))` `print(gross_sales(3, 450))`
Write two small functions, call each with at least two input sets, and show the difference between a function that prints and one that returns.

8. Expected result / what you should see

Different inputs produce expected results; returned value can be assigned/reused; no global variable is needed for the calculation.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Functions and scope: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Check three things separately: parameters, calculation, and return path. Temporarily print parameters at the function boundary if the wrong value is entering.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change. 3. Run the earliest independent control: For Functions and scope: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write `summarize_order(order_id, qty, unit_price)` returning a dict with `order_id` and `gross_sales`. Test two inputs.

13. Validate your result

For Functions and scope: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Different inputs produce expected results; returned value can be assigned/reused; no global variable is needed for the calculation.

14. Teach it back in your own words

Without notes, explain Functions and scope in your own words. Start with the purpose: Write small reusable functions with explicit parameters, returned values, predictable behavior and no accidental global dependencies. Then name one realistic failure mode and explain how you would prove the result using this check: For Functions and scope: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Functions and scope with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-08.

DE04-08 - Modules, imports and project structure

1. Why it matters

Split reusable helpers from an entry script, import them from the intended project root, and explain why folder/run location affects imports. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Split reusable helpers from an entry script, import them from the intended project root, and explain why folder/run location affects imports.

3. Prerequisite / what you already know

DE04-07. Local folder-based Python becomes required here if you used only a browser notebook earlier.

4. Picture it in your head

Modules are labeled toolboxes; imports bring selected tools into the current script. Project structure keeps code, configuration, data, and tests in predictable places.

5. New words before we use them

A Python module is a `.py` file. Importing lets one file use names defined in another file instead of duplicating code. A small project can have an entry script plus helper module; later modules will formalize larger package structure.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 3:12:46–3:28:16

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
# stage04_helpers.py def gross_sales(qty, unit_price): return qty * unit_price # M04_MODULE_DEMO_CANONICAL.py
from stage04_helpers import gross_sales print(gross_sales(2, 850)) Complete the two TODO helpers in
`stage04_helpers.py`, then run `M04_MODULE_DEMO_CANONICAL.py` from the practice-files folder.
```

8. Expected result / what you should see

Import succeeds from the documented folder; helper logic lives only in the helper module; running the entry file prints expected results.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: For Modules, imports and project structure: Inspect image/container/service state, command/exit status and the lesson-specific runtime proof (port response, persisted file, DNS/service reachability or configuration) from. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Confirm filenames and current working directory first. Run from the folder containing both files. Do not rename files halfway through without updating imports.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Modules are labeled toolboxes; imports bring selected tools into the current script. 3. Run the earliest independent control: For Modules, imports and project structure: Inspect image/container/service state, command/exit status and the lesson-specific runtime proof (port response, persisted file, DNS/service reachability or configuration) from a fresh run. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create `helpers.py` with one reusable calculation and a separate `run_demo.py` that imports it. Prove a clean run after closing/reopening the terminal/editor.

13. Validate your result

For Modules, imports and project structure: Inspect image/container/service state, command/exit status and the lesson-specific runtime proof (port response, persisted file, DNS/service reachability or configuration) from a fresh run.

14. Teach it back in your own words

Without notes, explain Modules, imports and project structure in your own words. Start with the purpose: Split reusable helpers from an entry script, import them from the intended project root, and explain why folder/run location affects imports. Then name one realistic failure mode and explain how you would prove the result using this check: For Modules, imports and project structure: Inspect image/container/service state, command/exit status and the lesson-specific runtime proof (port response, persisted file, DNS/service reachability or configuration) from.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Modules, imports and project structure with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-09.

DE04-09 - Exceptions and debugging basics

1. Why it matters

Read a traceback, isolate one defect at a time, and handle only expected/recoverable exceptions without hiding unrelated failures. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Read a traceback, isolate one defect at a time, and handle only expected/recoverable exceptions without hiding unrelated failures.

3. Prerequisite / what you already know

DE04-08.

4. Picture it in your head

An exception is a raised signal that normal execution cannot continue. Catch only errors you can handle; otherwise preserve the traceback because it points toward the failing line/state.

5. New words before we use them

A traceback shows the path of calls leading to an exception. Start at the final exception type/message, then find the first relevant line in your code. Fix one root cause and rerun so you know which change affected the behavior.

6. Useful video / source

Source: Corey Schafer — Python Try/Except Error Handling

Assigned scope: Full focused lesson

How to use it: Watch the focused exceptions lesson, type the examples, then debug the course task without the video.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=NIWwJbo-9_8)

URL: https://www.youtube.com/watch?v=NIWwJbo-9_8

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`raw = "12.5" try: qty = int(raw) except ValueError: print(f"Invalid integer quantity: {raw!r}")` Run ``M04_DEBUG_ME_CANONICAL.py`` unchanged. Save the first traceback. Repair one problem, rerun, and repeat until valid input works and invalid/missing values fail visibly.

8. Expected result / what you should see

First error is preserved; each fix is isolated; expected bad input has a useful message; no bare/broad swallowing of exceptions.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting "the script ran" as correctness. Your independent check is: For Exceptions and debugging basics: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

If you are changing many lines at once, restore the file and work one traceback at a time. If unsure what to catch, first let the error occur and identify its specific exception type.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An exception is a raised signal that normal execution cannot continue. 3. Run the earliest independent control: For Exceptions and debugging basics: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a `parse_quantity(raw)` function that returns an int for valid text and raises or reports a clear problem for invalid text without hiding unrelated exceptions.

13. Validate your result

For Exceptions and debugging basics: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: First error is preserved; each fix is isolated; expected bad input has a useful message; no bare/broad swallowing of exceptions.

14. Teach it back in your own words

Without notes, explain Exceptions and debugging basics in your own words. Start with the purpose: Read a traceback, isolate one defect at a time, and handle only expected/recoverable exceptions without hiding unrelated failures. Then name one realistic failure mode and explain how you would prove the result using this check: For Exceptions and debugging basics: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Exceptions and debugging basics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-10.

DE04-10 - Files, paths, CSV and JSON I/O

1. Why it matters

Build file paths predictably, read/write text and structured JSON, and inspect CSV/JSON without confusing file format with business grain. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Build file paths predictably, read/write text and structured JSON, and inspect CSV/JSON without confusing file format with business grain.

3. Prerequisite / what you already know

DE04-09.

4. Picture it in your head

Picture file I/O as opening a pipe between your program and durable storage: choose the correct path/mode/encoding, parse the format, close safely, then verify what was written.

5. New words before we use them

`pathlib.Path` represents filesystem paths without manual slash juggling. Build course paths from the project/script location when you need repeatability. Text files should use an explicit encoding such as UTF-8.

6. Useful video / source

Source: Corey Schafer — Python File Objects

Assigned scope: Full focused file I/O lesson; Teaching Book adds CSV/JSON specifics

How to use it: Use for file/path I/O mechanics. The course remains authoritative for CSV/JSON engineering patterns and validation.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=Uh2ebFW8OYM)

URL: <https://www.youtube.com/watch?v=Uh2ebFW8OYM>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
from pathlib import Path import json root = Path(__file__).parent out = root / "
run_manifest.json"
out.write_text(json.dumps({"
run_id"
:"
R1"
,"
rows"
:3}, indent=2), encoding="
utf-8"
) loaded = json.loads(out.read_text(encoding="
utf-8"
)) assert loaded["
rows"
] == 3 Use pathlib to locate the packaged retail CSV. Write a small JSON run manifest, read it back, and compare each
expected key/value.
```

8. Expected result / what you should see

Path does not depend on a random working folder; UTF-8 is explicit; input raw file remains unchanged; JSON read-back matches what was written.

9. Why it worked

The setup action is trustworthy because the actual file/system state is checked after the operation; successful clicking or command execution alone is not treated as proof. Your independent check is: For Files, paths, CSV and JSON I/O: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Print the resolved path before assuming the file is missing. If JSON parsing fails, inspect the exact file text and error location rather than rewriting it blindly.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture file I/O as opening a pipe between your program and durable storage: choose the correct path/mode/encoding, parse the format, close safely, then verify what was written. 3. Run the earliest independent control: For Files, paths, CSV and JSON I/O: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write JSON containing source filename, expected row count and status; reload and validate all three values. Also read the CSV header with the standard ``csv`` module.

13. Validate your result

For Files, paths, CSV and JSON I/O: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

14. Teach it back in your own words

Without notes, explain Files, paths, CSV and JSON I/O in your own words. Start with the purpose: Build file paths predictably, read/write text and structured JSON, and inspect CSV/JSON without confusing file format with business grain. Then name one realistic failure mode and explain how you would prove the result using this check: For Files, paths, CSV and JSON I/O: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Files, paths, CSV and JSON I/O with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-11.

DE04-11 - pandas foundations: load, inspect, filter and aggregate

1. Why it matters

Load a CSV into pandas, inspect shape/types, filter rows deliberately, calculate a column, aggregate at a stated grain and reconcile totals. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Load a CSV into pandas, inspect shape/types, filter rows deliberately, calculate a column, aggregate at a stated grain and reconcile totals.

3. Prerequisite / what you already know

DE04-10. Install pandas only now, at its first required use.

4. Picture it in your head

A DataFrame is a labeled table in memory. Load -> inspect schema/shape -> filter/transform -> aggregate is a pipeline; each step should have a row/column control.

5. New words before we use them

A pandas DataFrame is a table-like in-memory object. Start by inspecting shape, columns, sample rows and dtypes; do not immediately transform unfamiliar data. A boolean filter returns only rows meeting a condition.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 4:22:56–5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
import pandas as pd
df = pd.read_csv("M04_RETAIL_ORDERS_CANONICAL.csv")
df["sales"] = df["qty"] * df["unit_price"]
completed = df.loc[df["status"].eq("Completed")]
by_region = completed.groupby("region", as_index=False)["sales"].sum()
assert by_region["sales"].sum() == completed["sales"].sum()
```

`by_region.sum()` Inspect shape/columns/dtypes/head. Calculate sales. Filter Completed. Group by region. Save row counts and reconcile grouped sum to a separate Completed total.

8. Expected result / what you should see

Canonical controls: input rows=25; Completed rows=23; Completed sales=39320.00; missing sales_rep=1. Regional Completed totals: East 11700, North 9310, South 10210, West 8100.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: For pandas foundations: load, inspect, filter and aggregate: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared gr. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

If totals differ, compare in stages: input row count -> filter count -> calculated sales -> grouped total. Do not add `dropna` or `fillna` merely to make numbers match.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A DataFrame is a labeled table in memory. 3. Run the earliest independent control: For pandas foundations: load, inspect, filter and aggregate: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return Completed sales by region, top 3 Completed orders by sales, and rows with missing sales_rep. Write the result grain for each output.

13. Validate your result

For pandas foundations: load, inspect, filter and aggregate: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared group.

14. Teach it back in your own words

Without notes, explain pandas foundations: load, inspect, filter and aggregate in your own words. Start with the purpose: Load a CSV into pandas, inspect shape/types, filter rows deliberately, calculate a column, aggregate at a stated grain and reconcile totals. Then name one realistic failure mode and explain how you would prove the result using this check: For pandas foundations: load, inspect, filter and aggregate: Recompute one grouped total from detail rows and reconcile the sum of group totals to the known overall control; confirm the result has one row per declared gr.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for pandas foundations: load, inspect, filter and aggregate with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-12.

DE04-12 - Practical Big-O intuition

1. Why it matters

Recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime.

3. Prerequisite / what you already know

DE04-11.

4. Picture it in your head

Big-O is a growth sketch, not a stopwatch: ask how work grows when input size grows - constant, linear, logarithmic, quadratic - before worrying about tiny timing differences.

5. New words before we use them

Big-O describes how resource use grows as input size grows; it is not a stopwatch prediction. A single scan over n records is $O(n)$. A nested full scan can become $O(n^2)$.

6. Useful video / source

Source: freeCodeCamp/Jovian — Data Structures and Algorithms in Python

Assigned scope: 00:50:52 — Big-O complexity

How to use it: Watch only the Big-O block needed for practical $O(1)/O(n)/O(n^2)$ intuition; do not turn this into a competitive-programming detour.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=pkYVOMU3MgA)

URL: <https://www.youtube.com/watch?v=pkYVOMU3MgA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`seen = set()` for `order_id` in `["O1", "O2", "O2"]`: if `order_id` in `seen`: `print("duplicate", order_id)` `seen.add(order_id)` Compare repeated membership checks against a list and a set. Describe expected growth rather than timing one tiny example and declaring a universal winner.

8. Expected result / what you should see

Explanation correctly distinguishes list scan from average set lookup and labels a simple nested full scan as $O(n^2)$.

9. Why it worked

This works because recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime is made explicit before the result is accepted. Your independent check is: For Practical Big-O intuition: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Count how many times the inner operation may run as n grows. Ignore small constant factors at this conceptual level.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Big-O is a growth sketch, not a stopwatch: ask how work grows when input size grows - constant, linear, logarithmic, quadratic - before worrying about tiny timing differences. 3. Run the earliest independent control: For Practical Big-O intuition: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Explain why a set helps repeated duplicate-ID checks; write one nested-loop $O(n^2)$ example you would avoid for large data.

13. Validate your result

For Practical Big-O intuition: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Explanation correctly distinguishes list scan from average set lookup and labels a simple nested full scan as $O(n^2)$.

14. Teach it back in your own words

Without notes, explain Practical Big-O intuition in your own words. Start with the purpose: Recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime. Then name one realistic failure mode and explain how you would prove the result using this check: For Practical Big-O intuition: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Practical Big-O intuition with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-13.

DE04-13 - Coding drills: lists and strings

1. Why it matters

Solve small list/string problems from a blank function body, state edge cases and prove results with tests. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Solve small list/string problems from a blank function body, state edge cases and prove results with tests.

3. Prerequisite / what you already know

DE04-12. Keep answer keys/AI solving closed during the genuine attempt.

4. Picture it in your head

Treat the drill as manipulating a small sequence while preserving an invariant: know the input shape, perform one operation at a time, and check the exact output.

5. New words before we use them

Coding fluency grows by translating a requirement into input/output examples, a simple plan, code and tests. For these drills, the absence of a tutorial video is intentional. First write one or two examples and the plain-English steps you expect the function to perform.

6. Useful video / source

Internal teaching route

Internal teaching route. This lesson is fully taught in this book and its bundled practice. Use the mental model and worked demonstration here; no filler external video is required for mastery.

7. Follow with me once

Warm-up: write a plain-English algorithm for reversing word order while deciding how to treat extra spaces. Then implement only after the plan is saved.

8. Expected result / what you should see

Run ``M04_CODING_DRILLS_VISIBLE_TESTS.py``; supplied tests pass and you add at least one edge case of your own.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Coding drills: lists and strings: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Shrink the failing input. Print intermediate values. Return to the input/output contract before changing several lines at once.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat the drill as manipulating a small sequence while preserving an invariant: know the input shape, perform one operation at a time, and check the exact output. 3. Run the earliest independent control: For Coding drills: lists and strings: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Complete ``reverse_words`` and ``count_vowels`` in ``M04_CODING_DRILLS_CANONICAL.py``, then add one edge-case test for each.

13. Validate your result

For Coding drills: lists and strings: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Run ``M04_CODING_DRILLS_VISIBLE_TESTS.py``; supplied tests pass and you add at least one edge case of your own.

14. Teach it back in your own words

Without notes, explain Coding drills: lists and strings in your own words. Start with the purpose: Solve small list/string problems from a blank function body, state edge cases and prove results with tests. Then name one realistic failure mode and explain how you would prove the result using this check: For Coding drills: lists and strings: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Coding drills: lists and strings with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-14.

DE04-14 - Coding drills: dictionaries and sets

1. Why it matters

Use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements.

3. Prerequisite / what you already know

DE04-13.

4. Picture it in your head

Picture Python collections as different containers: list = ordered shelf, tuple = fixed record, set = unique-members box, dictionary = key-to-value lookup cabinet.

5. New words before we use them

A dictionary is a natural accumulator when each key needs a count or total. `counts[key] = counts.get(key, 0) + 1` is a common pattern. A set is natural when the question is only whether a value has been seen or which unique values exist.

6. Useful video / source

Internal teaching route

Internal teaching route. This lesson is fully taught in this book and its bundled practice. Use the mental model and worked demonstration here; no filler external video is required for mastery.

7. Follow with me once

`regions = ["East", "West", "East"]` `counts = {}` for region in regions: `counts[region] = counts.get(region, 0) + 1` `print(counts)`
Count three region labels with a dict and detect a repeated ID with a set. Explain why a nested loop is unnecessary.

8. Expected result / what you should see

Visible tests pass; output contract is deterministic; repeated duplicates are handled exactly as stated.

9. Why it worked

This works because use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements is made explicit before the result is accepted. Your independent check is: For Coding drills: dictionaries and sets: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Write the expected output for a tiny list by hand. If you need a count/total per key, use a dict accumulator; if only membership matters, start with a set.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture Python collections as different containers: list = ordered shelf, tuple = fixed record, set = unique-members box, dictionary = key-to-value lookup cabinet. 3. Run the earliest independent control: For Coding drills: dictionaries and sets: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Complete ``word_counts``, ``first_duplicate`` and ``sales_by_region`` in the drill file, then run visible tests and add one edge case each.

13. Validate your result

For Coding drills: dictionaries and sets: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Visible tests pass; output contract is deterministic; repeated duplicates are handled exactly as stated.

14. Teach it back in your own words

Without notes, explain Coding drills: dictionaries and sets in your own words. Start with the purpose: Use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements. Then name one realistic failure mode and explain how you would prove the result using this check: For Coding drills: dictionaries and sets: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Coding drills: dictionaries and sets with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-15.

DE04-15 - Stack, queue, search and sort basics

1. Why it matters

Recognize LIFO/FIFO behavior, implement linear search, explain binary-search sorted-data precondition and sort without accidentally mutating an input when preservation is required. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Recognize LIFO/FIFO behavior, implement linear search, explain binary-search sorted-data precondition and sort without accidentally mutating an input when preservation is required.

3. Prerequisite / what you already know

DE04-14.

4. Picture it in your head

Picture a stack of plates (last-in-first-out), a queue at a counter (first-in-first-out), and search/sort as strategies for finding or ordering items with different cost trade-offs.

5. New words before we use them

A stack removes the most recently added item first (LIFO). A queue removes the earliest added item first (FIFO); ``collections.deque`` supports efficient appends/pops from both ends. Linear search checks items until it finds a target or reaches the end.

6. Useful video / source

Source: freeCodeCamp/Jovian — Data Structures and Algorithms in Python

Assigned scope: 1:24:57 — binary vs linear search + 5:16:47 — sorting; course book adds light stack/queue scope

How to use it: Use only the mapped search/sort portions. Stack/queue remain a lightweight course concept; no giant DSA diversion.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=pkYVOMU3MgA)

URL: <https://www.youtube.com/watch?v=pkYVOMU3MgA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
from collections import deque
stack = []
stack.append("extract")
);
stack.append("transform")
) print(stack.pop())
q = deque(["job1", "job2"])
);
print(q.popleft())
values = [7,2,9,1,5];
ordered = sorted(values)
print(values, ordered)
Use a list as stack, deque as queue, run a linear-search function and compare `sorted` vs `.sort()` behavior.
```

8. Expected result / what you should see

LIFO/FIFO outputs are correct; original list is preserved when required; linear search returns index/-1; binary search precondition is stated.

9. Why it worked

The spreadsheet result is trustworthy because the source structure/formula logic is preserved and an independent cell/row/total check proves the displayed result came from the intended calculation. Your independent check is: For Stack, queue, search and sort basics: record the actual before/after state and verify the lesson requirement with an independent control. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Act out the structure with 3 labels. For search, write what happens to each candidate position. For sorting, print both original and result.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture a stack of plates (last-in-first-out), a queue at a counter (first-in-first-out), and search/sort as strategies for finding or ordering items with different cost trade-offs. 3. Run the earliest independent control: For Stack, queue, search and sort basics: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Push extract/transform/load and pop once; create a FIFO queue example; sort [7,2,9,1,5] without mutating original; implement `linear_search`.

13. Validate your result

For Stack, queue, search and sort basics: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: LIFO/FIFO outputs are correct; original list is preserved when required; linear search returns index/-1; binary search precondition is stated.

14. Teach it back in your own words

Without notes, explain Stack, queue, search and sort basics in your own words. Start with the purpose: Recognize LIFO/FIFO behavior, implement linear search, explain binary-search sorted-data precondition and sort without accidentally mutating an input when preservation is required. Then name one realistic failure mode and explain how you would prove the result using this check: For Stack, queue, search and sort basics: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Stack, queue, search and sort basics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE04-16.

DE04-16 - Integrated Python checkpoint and reproducible script habits

1. Why it matters

Combine core Python skills into a repeatable folder-based program with explicit input validation, deterministic output, visible failure behavior and a documented run command. This matters because Python can run successfully while using the wrong state, type or control flow, so observable output must be checked.

2. What you are learning

Combine core Python skills into a repeatable folder-based program with explicit input validation, deterministic output, visible failure behavior and a documented run command.

3. Prerequisite / what you already know

DE04-01.15 evidence complete. Keep the PyPA reference closed until the first checkpoint attempt is saved.

4. Picture it in your head

The checkpoint is a miniature software handoff: a fresh user should be able to run the script from a clean state, reproduce the output, and understand errors without editing hidden local assumptions.

5. New words before we use them

A reproducible script can be run again from known inputs and produce explainable outputs without hidden notebook state. Keep inputs explicit, split logic into small functions, validate required fields before calculating, fail visibly when the input contract is broken, and write deterministic output. Project structure communicates how to rerun the work.

6. Useful video / source

Source: Python Packaging User Guide — Packaging Python Projects

Assigned scope: Current project layout: README, pyproject.toml, src/, tests/; use as a structure reference after your first checkpoint attempt

How to use it: Attempt the integrated checkpoint first. Use this current PyPA guide only to repair reproducibility/project-structure weaknesses; it is not a solution walkthrough.

[OPEN VIDEO / SOURCE - CLICK HERE \(packaging.python.org\)](https://packaging.python.org/en/latest/tutorials/packaging-projects/)

URL: <https://packaging.python.org/en/latest/tutorials/packaging-projects/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

No answer walkthrough. Read the TODO/rubric, state the input/output contract, then implement one function at a time.

8. Expected result / what you should see

Clean run succeeds; controls reconcile; bad/missing input fails visibly; raw input is unchanged; output is deterministic; documented run command works from a fresh terminal.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: For Integrated Python checkpoint and reproducible script habits: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected. The proof is the program output/state on a fresh input, not the absence of an exception.

10. Common beginner mistakes

Split failures by boundary: path -> parse -> required columns -> calculation/filter -> aggregation -> output write -> rerun. Repair one boundary at a time.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: The checkpoint is a miniature software handoff: a fresh user should be able to run the script from a clean state, reproduce the output, and understand errors without editing hidden local assumptions. 3. Run the earliest independent control: For Integrated Python checkpoint and reproducible script habits: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Complete the checkpoint: load/validate retail CSV, calculate sales, keep Completed, produce regional totals + top 3 + missing-rep count, write deterministic JSON, prove one bad-input failure, and document the run command.

13. Validate your result

For Integrated Python checkpoint and reproducible script habits: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Integrated Python checkpoint and reproducible script habits in your own words. Start with the purpose: Combine core Python skills into a repeatable folder-based program with explicit input validation, deterministic output, visible failure behavior and a documented run command. Then name one realistic failure mode and explain how you would prove the result using this check: For Integrated Python checkpoint and reproducible script habits: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Integrated Python checkpoint and reproducible script habits with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.